

Document number	P3218R0
Date	2024-04-09
Reply-to	Jarrad J. Waterloo <descender76 at gmail dot com>
Audience	Evolution Working Group (EWG) Core Working Group (CWG)

const references to constexpr variables

Table of contents

- const references to constexpr variables
 - Abstract
 - Simpler
 - Safer
 - Relationship to constexpr wrapper
 - Related Work
 - Summary
 - Frequently Asked Questions
 - References

Abstract

This proposal is an addendum to the "constexpr structured bindings and references to constexpr variables" ^[1] proposal with a focus on simplicity and safety.

Simpler

The P2686 , constexpr structured bindings and references to constexpr variables ^[1:1], proposal outlines 4 solutions to its problem.

1. "Allowing static and non-tuple constexpr structured binding"
2. "Making constexpr implicitly static"
3. "Always re-evaluate a call to get?"

4. "Symbolic addressing"

Of these four, P2686 ^[1:2] recommends that the first and the last be implemented. Concerning the first, the authors wrote; *"Independently of the other solutions presented here, this option would be useful and should be done"*. Concerning the last, the authors wrote; *"The most promising option - the one we think should be pursued"*.

This proposal does not argue for or against the first and the last. To the contrary, this proposal argues that there is a place in at least some [limited] capacity for the second solution, Making `constexpr` implicitly `static`.

given

```
struct point
{
    int x;
    int y;
};

const point& dangler(const point& p)
{
    // ...
    return p;
}

const std::string& dangler(const std::string& s)
{
    // ...
    return s;
}
```

usage

```
// {4, 2} is static
const point& always_safe = {4, 2};
const point& safe = dangler({4, 2});
// "42"s is static
const std::string& always_safe = "42"s;
const std::string& safe = dangler("42"s);
```

NOTE: While this proposal is only concerned about "const references to constant expressions" implicitly creating constants, it would make sense to end programmers for `const` variables as well.

<pre>// {4, 2} is static const point& always_safe = {4, 2};</pre>	<pre>// {4, 2} can be static const point always_safe{4, 2};</pre>
<pre>// "42"s is static const std::string& always_safe = "42"s;</pre>	<pre>// "42"s can be static const std::string always_safe{"42"}</pre>



It should be noted that `constexpr` structured bindings and references to `constexpr` variables 's^[1:3] first option is simpler than the last, at least in wording but is more complex than the last in the sense that it explicitly requires the programmer to litter their code with `static`. The second option is simpler than both the first and the last options because the `static` is implied, just as it is at namespace scope, resulting in simplified wording and simplified usage by the end programmers. Are the reasons presented against the second option totally legitimate or could those rare exceptions be handled in such a way that simplicity wouldn't be lost for the greater whole?

Let's consider the complete rationale against, before breaking down each point in turn.

constexpr structured bindings and references to constexpr variables ^[1:4]

"1. Making constexpr implicitly static"

"We could make constexpr variables implicitly static, but doing so would most certainly break existing code, in addition to being inconsistent with the meaning of constexpr :"

```
int f() {
    constexpr struct S {
        mutable int m ;
    } s{0};
    return ++s.m;
}

int main() {
    assert(f() + f() == 2); // currently 2. Becomes 3 if 's' is made implicitly
}
```

"So this solution is impractical. We could make constexpr static only in some cases to alleviate the breakages or even make only constexpr bindings static, not other variables, but this, feels like a hack rather than an actual solution."

"doing so would most certainly break existing code"

To the contrary, code that is broken is now fixed. Code that would be invalid is now valid, makes sense and can be rationally explained. Let me explain. This feature not only changes the point of destruction but also the point of construction. Instances that were of automatic storage duration, are now of static storage duration. Instances that were temporaries, are no longer temporaries. Surely, something must be broken! The C++ standard already recognized that there are other opportunities for constant initialization.

Working Draft, Standard for Programming Language C++ ^[2]

"6.9.3.2 Static initialization [basic.start.static]"

"3 An implementation is permitted to perform the initialization of a variable with static or thread storage duration as a static initialization even if such initialization is not required to be done statically ..."

So, what is the point? For the instances that would benefit from implicit constant initialization, there are currently NO guarantees as far as their lifetime and as such is indeterminate. With this portion of the proposal, a guarantee is given and as such that which was non-determinate becomes determinate.

It should also be noted that while this enhancement is applied implicitly, programmers have opted into this up to three times.

1. The programmer of the type must have provided a means for the type to be constructed at compile time likely by having a `constexpr` constructor.
2. The programmer of the variable or function parameter must have stated that they want a `const`.
3. The end programmer has `constexpr` the variable or argument.

Having expressed constant requirements three times, it is pretty certain that the end programmer wanted a constant, even if it is anonymous.

Next, a function that takes an argument by reference does not know whether the caller of said function will pass to it a global, local, temporary or a dynamically created object. Rather it expects that said parameter would exist for the life of the function and if the reference is returned, in part or in whole, than the life of the object needs to be longer.

"inconsistent with the meaning of `constexpr`"

Really! One only has to read the original motivations for `constexpr` to realize that `static` is consistent with its meaning.

n1511

2003

"This note proposes a notion of user-defined literals based on literal constructors without requiring new syntax. If combined with the separate proposal for generalized initializer lists, it becomes a generalization of the C99 notion of compound literals."

*"However, a constructor is a very general construct and there have been many requests for a way to express literals for user-defined types in such a way that a programmer can be **confident that a value will be constructed at compile time and potentially stored in ROM**. For example:"*

```
complex z(1,2); // the variable z can be constructed at compile time
const complex cz(1,2); // the const cz can potentially be put in ROM
```

"Personally, I prefer (1): basically, a value is a literal if it is composed out of literals and implemented by a literal constructor. The problem with that is that some people will not trust compilers to do proper resolution, placement in ROM, placement in text segment"

"C99 compound literals"

"In C99, it is explicitly allowed to take the address of a compound literal. For example:"

```
f(&(struct foo) { 1,2 } );
```

"This makes sense only if we assume that the {1,2} is stored in a data segment (like a string literal, but different from a int literal). I see no problem allowing that"

"It would be tempting to expand this rule to user-defined literals bound to references."

2007

"This paper generalizes the notion of constant expressions to include constant-expression functions and user-defined literals"

*"The goal is ... to **increase C99 compatibility.**"*

...

*"**simplify the language** definition in the area of constant expression **to match existing practice**"*

...

*"**3. Static initialization of objects with static storage.**"*

*"... However **we strive for more uniform and consistency among related C++ language features and compatibility**"*

*"**3 Problems**"*

...

*"**3.4 Unexpected dynamic initialization**"*

*"However, it is possible to be surprised by expressions that (to someone) **"look const"** but are not."*

*"**3.5 Complex rules for simple things**"*

*"The focus of this proposal is to address the issues mentioned in preceding sections. However, discussions in the Core Working Group at the Berlin meeting (April 2006) concluded that the current rules for integral constant expressions are too complicated, and source of several Defect Reports. **Consequently, a "cleanup", i.e. adoption of simpler, more general rules is suggested.**"*

...

"As for other const variables, storage need not be allocated for a constant expression datum, unless its address is taken."

```
// the &x forces x into memory
```

"When the initializer for an ordinary variable (i.e. not a constexpr) happens to be a constant, the compiler can choose to do dynamic or static initialization (as ever)."

"Declaring a constructor constexpr will help compilers to identify static initialization and perform appropriate optimizations (like putting literals in read-only memory.)"

Note that since "ROM" isn't a concept of the C++ Standard and what to put into ROM is often a quite subtle design decision, this proposal simply allows the programmer to indicate what might be put into ROM (constant-expression data) rather than trying to specify what actually goes into ROM in a particular implementation."

"We do not propose to make constexpr a storage-class-specifier because it can be combined with either static or extern or register, much like const."

One of the main motivations for `constexpr` was for objects to be placed in ROM . What is read only memory, if not `const` and static storage duration!

"the mutable example"

```
int f() {
    constexpr struct S {
        mutable int m ;
    } s{0};
    return ++s.m;
}

int main() {
    assert(f() + f() == 2); // currently 2. Becomes 3 if 's' is made implicitly st
}
```

It's a shame that `mutable` is permitted to be used in constant expressions as it is rarely used, actively discouraged and potentially disabled by future safety enhancements. It is barely rational to have mutation associated with something associated with ROM as that typically brings about segmentation faults. It is also unnatural since template, `constexpr` and `constexpr` functions are C++'s closest thing to pure functions. One can hardly say in the example `f()` is pure if it sometimes return 1 and another time returns 2, given the same arguments. The damage is done but also mostly irrelevant, with respect to this proposal. We'll see this in the next point but before moving on to it, keep in mind that the example is not an example of taking a reference to `constexpr` variable, more less a `const` reference to `constexpr` variable.

"We could make constexpr static only in some cases to alleviate some of the breakages or even make only constexpr bindings static, not other variables, but this option feels like a hack rather than an actual solution."

Actually, no it doesn't. It is rather consistent with what is already proposed for C++26.

P2752R3

Static storage for braced initializers ^[5]

4. Mutable members

...

Vendors are expected to deal with this by simply disabling their promote-to-shared-storage when the element type (recursively) contains any mutable bits.

...

7. Proposed wording relative to the current C++23 draft

...

[Example 12:

```
void f(std::initializer_list<double> il);
void g(float x) {
    f({1, x, 3});
}
void h() {
    f({1, 2, 3});
}

struct A {
    mutable int i;
};
void q(std::initializer_list<A>);
void r() {
    q({A{1}, A{2}, A{3}});
}
```

The initializations can be implemented in a way roughly equivalent to this:

```

void g(float x) {
    const double __a[3] = {double{1}, double{x}, double{3}}; // backing array
    f(std::initializer_list<double>(__a, __a+3));
}
void h() {
    static constexpr double __b[3] = {double{1}, double{2}, double{3}}; // bac
    f(std::initializer_list<double>(__b, __b+3));
}
void r() {
    const A __c[3] = {A{1}, A{2}, A{3}}; // backing array
    q(std::initializer_list<A>(__c, __c+3));
}

```

This already accepted solution to the mutable problem could also be used as the solution to the mutable problem of this proposal.

Safer

Besides simplicity, why does this proposal recommend that const references to constexpr variables have static storage duration? One word, safety. These objects would be impossible to dangle and also would be thread safe. This also fixes one of C++'s most embarrassing forms of dangling; dangling constants. This problem is unique to C++. Practically all languages, except C++, doesn't immediately dangle their constants, not even assembly, C, or even the earliest version of C++, cfront.

Things are even worse in C++ because as programmers we can't look at the code and know for sure whether an object is static or not. This requires C++ programmers even beginners to have to look at machine code to see how the compiler decided where to store the object. Not only does this vary from compiler to compiler, it also varies within any given compiler. Frequently in optimized release builds these constant objects are given static duration but made a dangling local in debug builds. This is totally backwards, as the increased safety is expected in debug builds.

Further if the optimized build is both the safest and the better performant then why shouldn't it be available always! Dangling constants is like returning from a function using an input parameter in pre C++11 because programmers did not have sufficient assurance that "return value optimization" would occur.

Similarly, programmers have to name the currently unnamed temporary and move it far from the point of use, just to ensure dangling doesn't occur in both debug and release builds. This habit works against the recommendation to use unnamed [temporary] variables and move semantics.

Relationship to constexpr wrapper

This proposal is related to the `std::constexpr_wrapper` ^[6] proposal on the safety level. While that proposal explicitly creates safe anonymously named constants as a library, this proposal would be an implicit language feature. As a language feature, there are a couple of advantages.

- 1. `std::constexpr_wrapper` ^[6:1] only works with structural types as those are currently the only types permitted for non type template parameters. This proposal can work with all types that haven't deleted their reference of operators.
- 2. simpler and more consistent, not having to resort to different libraries for structural and non structural types

	this proposal	<code>std::constexpr_wrapper</code>
structural type	<pre>// {4, 2} is static const point& safe = dangler({4, 2});</pre>	<pre>// {4, 2} is a temp const point& safe</pre>
non structural type	<pre>// "42"s is static const std::string& safe = dangler("42"s);</pre>	<pre>// "42"s is a temp const std::string& [] /*constexpr eval { static constexpr return and }());</pre>

As demonstrated in the previous examples, the result of this proposal is that the same simple syntax is used regardless of whether the type was structural or not, while the current state of affairs require using a lambda function in order to provide an anonymously named constant.

Related Work

A similar but more complicated feature exists in the Rust programming language known as `rvalue static promotion` ^[7]. This makes sense for Rust as all types are by default movable. Similarly, the proposed feature would work for C++ since by default all types are referable.

```
let x: &'static u32 = &42;
```

The proposed feature would be simpler than `rust` as it doesn't require an explicit `'static` lifetime but rather would be implied because the would be temporary was `constexpr` capable, initialized with only constants and was initially assigned to a `const &`.

More verbose information on the motivation for this proposed feature also exists in the previous `constant dangling` ^[8] proposal.

Summary

"Making `constexpr` implicitly static" ^[1:5] for constants makes the language simpler and at least for constant references makes the language safer. Like the `Static storage for braced initializers` ^[5:1] proposal it would minimize "CPU cycles", "stack space" and heap allocations caused by types that dynamically allocate such `std::string` and `std::vector` when they are better off being a constant.

Frequently Asked Questions

Who would even use these features? Their isn't sufficient use to justify these changes.

The best proof can be found in our usage and other proposals.

C++ Core Guidelines F.16: For "in" parameters, pass cheaply-copied types by value and others *by reference to const* ^[9]

In C++, we use `const` parameters alot. This is the first of three requirements of `implicit constant initialization`. What about the use of types that can be constructed at compile time?

- **C++20**: `std::pair`, `std::tuple`, `std::string`, `std::vector`
- **C++23**: `std::optional`, `std::variant`, `std::unique_ptr`

As there was sufficient use to justify making the constructors of any one of these listed above types to be `constexpr` than there would be sufficient use of the `implicit constant initialization` feature which would use them all as this satisfies its first and second of the three requirements that the instances be constructable at compile time and that they `const`.

1. **The programmer of the type must have provided a means for the type to be constructed at compile time likely by having a `constexpr` constructor.**
2. **The programmer of the variable or function parameter must have stated that they want a `const`.**
3. The end programmer have `const-initialized` the variable or argument.

Doesn't this make C++ harder to teach?

Until the day that all dangling gets fixed, any incremental compile time fixes to dangling still would require programmers to be able to identify any remaining dangling and know how to fix it specific to the given scenario, as there are multiple solutions. Since dangling occurs even for things as simple as constants then dangling resolution still have to be taught, even to beginners. As this proposal fixes these types of dangling, it makes teaching C++ easier because it makes C++ easier.

So, what do we teach now and what bearing does these teachings, the C++ standard and this proposal have on one another.

C++ Core Guidelines

F.42: Return a `T*` to indicate a position (only) ^[10]

Note Do not return a pointer to something that is not in the caller's scope; see F.43. ^[11]

Returning references to something in the caller's scope is only natural. It is a part of our reference delegating programming model. A function when given a reference does not know how the instance was created and it doesn't care as long as it is good for the life of the function call (and beyond). Unfortunately, scoping temporary arguments to the statement instead of the containing block doesn't just create immediate dangling but it provides to functions references to instances that are near death. These instances are almost dead on arrival. Having the ability to return a reference to a caller's instance or a sub-instance thereof assumes, correctly, that reference from the caller's scope would still be alive after this function call. The fact that temporary rules shortened the life to the statement is at odds with what we teach. This proposal extends the lifetimes of some temporaries, making them global which is not only natural but also consistent with what programmers already know. It is also in line with what we teach as was codified in the C++ Core Guidelines.

Other types of dangling can still occur. One simple type is directly called out in the C++ Core Guidelines.

C++ Core Guidelines

F.43: Never (directly or indirectly) return a pointer or a reference to a local object

[11:1]

Reason *To avoid the crashes and data corruption that can result from the use of such a dangling pointer.* [11:2]

...

Note *This applies only to non-static local variables. All static variables are (as their name indicates) statically allocated, so that pointers to them cannot dangle.* [11:3]

Other than turning some of these locals into globals, this proposal does not solve nor contradict this teaching. If anything, by cleaning up, the remaining dangling is made all the more visible.

Further, what is proposed is easy to teach because we already teach it and it makes C++ even easier to teach.

- We already teach that native string literals don't dangle because they have static storage duration. This proposal just extends the concept to constants, as expected. This increases good consistency and reduces a bifurcation that is currently taught.
- Somehow, we already teach the temporary lifetime extension rules which consist of numerous paragraphs and exception examples. This is just another lifetime extension but instead of extending an object's lifetime from being scoped to the statement to instead be scoped globally and not the block.

All of this can be done without adding any new keywords or any new attributes. We just use constant concepts that beginners are already familiar with. In fact, we will be working in harmony with all that we already teach about globals in the C++ Core Guidelines [12].

- I.2: Avoid non-const global variables [13]
- I.22: Avoid complex initialization of global objects [14]
- F.15: Prefer simple and conventional ways of passing information [15]
- F.16: For “in” parameters, pass cheaply-copied types by value and others by reference to const [16]
- F.43: Never (directly or indirectly) return a pointer or a reference to a local object [11:4]

- R.5: Prefer scoped objects, don't heap-allocate unnecessarily [17]
- R.6: Avoid non-const global variables [18]
- CP.2: Avoid data races [19]
- CP.24: Think of a thread as a global container [20]
- CP.32: To share ownership between unrelated threads use `shared_ptr` [21]

References

1. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p2686r3.pdf> (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p2686r3.pdf>)      
2. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/n4910.pdf> (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/n4910.pdf>) 
3. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2003/n1511.pdf> (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2003/n1511.pdf>) 
4. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2007/n2235.pdf> (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2007/n2235.pdf>) 
5. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2752r3.html> (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2752r3.html>)  
6. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p2781r4.html> (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p2781r4.html>)   
7. https://rust-lang.github.io/rfcs/1414-rvalue_static_promotion.html (https://rust-lang.github.io/rfcs/1414-rvalue_static_promotion.html) 
8. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2724r1.html> (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2724r1.html>) 
9. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#Rf-in> (<https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#Rf-in>) 

10. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#f42-return-a-t-to-indicate-a-position-only> (<https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#f42-return-a-t-to-indicate-a-position-only>) 
11. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#f43-never-directly-or-indirectly-return-a-pointer-or-a-reference-to-a-local-object>
(<https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#f43-never-directly-or-indirectly-return-a-pointer-or-a-reference-to-a-local-object>)     
12. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines>
(<https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines>) 
13. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#i2-avoid-non-const-global-variables> (<https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#i2-avoid-non-const-global-variables>) 
14. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#i22-avoid-complex-initialization-of-global-objects> (<https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#i22-avoid-complex-initialization-of-global-objects>) 
15. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#f15-prefer-simple-and-conventional-ways-of-passing-information>
(<https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#f15-prefer-simple-and-conventional-ways-of-passing-information>) 
16. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#f16-for-in-parameters-pass-cheaply-copied-types-by-value-and-others-by-reference-to-const>
(<https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#f16-for-in-parameters-pass-cheaply-copied-types-by-value-and-others-by-reference-to-const>) 
17. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#r5-prefer-scoped-objects-dont-heap-allocate-unnecessarily>
(<https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#r5-prefer-scoped-objects-dont-heap-allocate-unnecessarily>) 

18. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#r6-avoid-non-const-global-variables> (<https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#r6-avoid-non-const-global-variables>) 
19. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#cp2-avoid-data-races> (<https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#cp2-avoid-data-races>) 
20. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#cp24-think-of-a-thread-as-a-global-container> (<https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#cp24-think-of-a-thread-as-a-global-container>) 
21. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#cp32-to-share-ownership-between-unrelated-threads-use-shared_ptr (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#cp32-to-share-ownership-between-unrelated-threads-use-shared_ptr) 